

Prompting for Performance: Profile-Guided LLM Optimization of Real GPU Applications

Anonymous ACL submission

Abstract

We present an empirical study of how prompt design—specifically, the task-specific guardrails and the profiling evidence placed in the prompt—shapes the ability of large language models to optimize GPU code, and of how far the resulting gains survive on production scientific applications. Our framework closes the loop from profiling to prompting, generation, build, correctness verification and timed measurement, with bounded repair from compiler and runtime feedback. We first ablate prompt components across six variants, two models and six HPC kernels: stripping contextual framing consistently degrades optimization quality, whereas an explicit reasoning step is the only variant that never hurts. We then show that guardrails are not merely a safety concern but a correctness one. Without an explicit semantic-preservation constraint, both models optimize a benchmark by deleting the very computation it exists to measure, reporting speedups of $1504\times$ and $2809\times$ while the program’s output and its built-in verification remain unchanged; the constraint reduces both to honest results. Moving from kernels to applications—AutoDock-GPU, LAMMPS and Cholla, each gated by its own scientific output and measured under interleaved timing on shared GPUs—we find that models reliably discover local, exact transformations and reliably miss cross-cutting restructuring: the best model output reaches $1.571\times$ on Cholla, beating our expert hand-tuned reference, while on AutoDock-GPU no model or prompt variant found the reduction fusion worth $1.532\times$ by hand. Finally, an ablation that removes profiling data from an otherwise identical prompt shows that PC-sampling and roofline context is worth 7.7–11.5% on the large application and nothing measurable on single-kernel benchmarks, indicating that its value scales with how hard the bottleneck is to locate from source alone. We also report failure modes that appear only on

real code—compilation-context blindness, applications that discard the diagnostics a repair loop needs, and shared-machine measurement pitfalls—and argue that benchmark-only evaluation overstates the readiness of LLM-driven performance tools.

1 Introduction

GPU performance engineering is expert work. Getting a kernel to run faster requires reasoning about the memory hierarchy, warp scheduling, occupancy and instruction-level behaviour at the same time, and expert tuners routinely spend weeks on a single kernel. Large language models can attempt the same edit in seconds, which raises an obvious question: how much of that expertise can be supplied through the prompt, and how much of the resulting speedup survives contact with a real application?

Existing code benchmarks do not answer this. HumanEval-style suites (Chen et al., 2021) and repository-level issue resolution (Jimenez et al., 2024) score functional correctness, not execution time, and recent GPU-specific benchmarks (Ouyang et al., 2025; Nichols et al., 2024) evaluate standalone kernels written from a reference implementation rather than edits to code that a scientific application already depends on. Meanwhile, the first thing a human expert looks at—profiler output—is exactly the kind of context an LLM never sees by default.

This paper studies an end-to-end loop that closes that gap: profile the application, summarize the profile into the prompt, ask the model for an optimized version of one source file, build it, verify it against the application’s own notion of correctness, measure it, and feed failures back for a bounded number of repair iterations. Within this loop we treat the prompt as the object of study. Our contributions are:

- A **prompt-component ablation** over six

086	variants, two models and six HeCBench kernels (Section 3–5), showing that prompt components are not equally important: stripping context (minimal) consistently degrades results, while an explicit reasoning step (cot) is the only variant that never hurts.	135
087		136
088		137
089		138
090		139
091		140
092	• A semantic-preservation constraint that suppresses benchmark gaming (Section 6). Without it, both models delete the very computation a benchmark exists to measure and report speedups of $1504\times$ and $2809\times$ while output and the built-in verification remain unchanged; with it, both return honest results. We also report the detection rule we use, since neither output comparison nor the benchmark’s own check can catch this class of edit.	141
093		142
094		143
095		144
096		145
097		146
098		147
099		148
100		
101		
102		
103	• An end-to-end evaluation on three production applications —AutoDock-GPU, LAMMPS and Cholla (Section 7)—with per-application correctness gates that we validate by fault injection, and interleaved measurement on shared GPUs. The best model output reaches $1.571\times$ on Cholla, beating our expert hand-tuned reference, while on AutoDock-GPU no model or prompt found the restructuring that is worth $1.532\times$ by hand.	149
104		150
105		151
106		152
107		153
108		154
109		155
110		156
111		157
112		158
113		159
114	• A profile-data ablation (Section 8) that isolates the contribution of PC-sampling and roofline data by removing it from an otherwise identical prompt. Profiling data is worth 7.7–11.5% on the large application and nothing measurable on single-kernel benchmarks, which suggests its value scales with how hard the bottleneck is to locate from source alone.	160
115		161
116		162
117		163
118		164
119		165
120		166
121		167
122	• A measurement of what happens when profiling data is made prescriptive (Section 9). Converting metrics into a checklist of named remedies produces both our largest gain ($1.04\times \rightarrow 1.38\times$) and our largest loss ($7.84\times \rightarrow 1.08\times$, where the checklist displaced a restructuring the model had found unaided), at repetition spreads below 5%. Interpretation widens the distribution of outcomes rather than shifting its centre, which a mean speedup hides.	168
123		169
124		170
125		171
126		172
127		173
128		174
129		175
130		176
131		177
132		178
133	• A set of failure modes specific to real code (Section 7.5), including compilation-context blindness (the model cannot see that its file is textually included elsewhere, or that ten call sites depend on a signature), applications whose error handling discards the diagnostic the repair loop needs, and measurement pitfalls on shared machines that silently invalidate results.	179
134		180

Taken together, the results argue that prompt design is a first-order lever for turning general-purpose code models into usable performance tools—but also that conclusions drawn on single-kernel benchmarks do not automatically transfer to the applications those kernels are meant to represent.

2 Methodology

2.1 Pipeline

The framework takes a source file, a build command and a run command, and closes the loop *profile* $\rightarrow$ *prompt* $\rightarrow$ *generate* $\rightarrow$ *build* $\rightarrow$ *verify* $\rightarrow$ *measure*, with bounded repair. The optimized file returned by the model replaces the original in a private copy of the application tree, so the build, the callers and the input data are those of the real program. If the build or the correctness check fails, the error summary is appended to the prompt and the model is asked to repair; we allow three iterations before recording the run as a failure.

2.2 Profiling data

We profile with Nsight Compute in two passes: an uncapped pass over every launch that yields true launch counts and per-launch durations, and a capped `--set full` pass that yields the detail pages. From the resulting reports we derive three kinds of prompt context:

- **PC sampling** (`--page source`): per-source-line warp stall reasons, which we aggregate to the lines with the highest stall ratios.
- **Roofline** (`--page details`): per-kernel throughput relative to DRAM and compute peaks, plus the analysis rules Nsight Compute reports.
- **Importance analysis**: per-kernel aggregate cost (launch count $\times$ per-launch time) used to rank which kernel is worth editing at all.

A capped profile is only representative if the launches it keeps are: in one application the first

200 launches were setup rather than simulation, and we skip a measured number of launches before profiling (Section 7).

2.3 Prompt structure

The prompt has five regions: (i) the source file, (ii) the profile summaries above, (iii) hardware-specific guardrails (target architecture, shared-memory limits, unavailable intrinsics), (iv) general pitfall-avoidance guardrails (entry-point preservation, no signature changes, no new top-level symbols, semantic preservation), and (v) an output-format specification requiring a single fenced code block plus structured lists of applied and rejected optimizations. Section 3 ablates these regions; Section 8 removes region (ii) entirely.

2.4 Correctness

Correctness is checked in whichever of two ways the program supports. If the program prints numerical results, the optimized version must reproduce the baseline’s numbers; if it ships a built-in verification, we use its verdict, tailored per application. For the production applications neither is sufficient on its own, so we construct explicit gates—bit-exact field comparison where the application is deterministic, and a statistical band where it is not—and validate each gate by injecting a deliberate fault and confirming the gate rejects it (Section 7).

2.5 Timing

Time comes from the program’s own reported kernel or phase time wherever it provides one, and from wall clock otherwise; for the applications this matters, since AutoDock-GPU’s CUDA setup alone is 68 s on a cold driver and would swamp a few percent of solver time. The number reported by the pipeline during a run is indicative only. Every number we report comes from a separate measurement pass that builds each arm once, discards a warmup run, interleaves all arms of an application round-robin within a single session, waits for an idle GPU before starting, and logs other users’ GPU processes at the start *and* the end of every repetition. We take medians over the repetitions that were clean by that test—seven repetitions per arm for the applications, five for the profile-data ablation on benchmarks—and treat repetitions that disagree with their neighbours as suspect even when no foreign process was seen. Between-session drift on these shared nodes reached 49%

on identical code, which is an order of magnitude larger than most of the effects we are trying to measure.

2.6 Workload selection

The benchmark study spans memory-bound (shmembench) and compute-bound (bitonic-sort, md5hash, atan2) kernels. The application study selects, for each program, the single file containing the kernel that dominates GPU time, and—critically—only after confirming that this kernel also dominates *application* time.

3 Ablation

The full prompt is not a monolith: it bundles a persona, hardware-specific guardrails, a set of prohibitions, a formal-equivalence requirement, a request for an explanation of each change, and an output-format specification. To find out which of these actually matter, we evaluate six variants, each of which removes or adds exactly one component while leaving the source code and the profiling data in place. Two further variants (no_format, no_hardware) are implemented but excluded from the measured grid, for the reasons noted below.

No Proof (no_proof). The full prompt requires the model to state preconditions, postconditions and loop invariants, and to argue that the optimized kernel is semantically equivalent to the original. Removing this requirement lets the model apply a transformation without having to justify it. The consequence is visible on shmembench, whose swap loop preserves an algebraic sum invariant: without the proof requirement, Gemini 2.5 Pro exploits that invariant to eliminate shared-memory traffic altogether and reports a $213\times$ apparent speedup that reflects no hardware improvement. GPT-5 applies the same transformation under every variant, including the full prompt—which is what motivates the explicit semantic-preservation constraint of Section 6.

No Constraints (no_constraints). The full prompt carries a numbered list of prohibitions: do not rename or remove the entry point, do not change kernel signatures, do not introduce new top-level symbols, do not trade correctness for speed. This variant deletes every such line, giving the model unconstrained freedom to restructure. The effect is model-dependent rather than

uniformly good or bad: it produces GPT-5’s best bitonic-sort result ($1.14\times$) and one of Gemini’s worst ($0.90\times$).

Minimal (minimal). This variant replaces the entire instruction block—persona, guardrails, prohibitions, proof requirement, explanation requirement and format specification—with a single sentence: “*Optimize the following CUDA code for execution time.*” The source code and the profiling summaries are still supplied, so minimal isolates the value of the *instructions* rather than of the evidence; removing the evidence instead is the separate ablation of Section 8. Minimal is the only variant that degrades both models: on md5hash GPT-5 produces code 51% slower than its best variant, and on shmembench Gemini produces code roughly four times slower than under any other variant.

No Explanation (no_explanation). The full prompt asks the model to return, alongside the code, a structured list of the optimizations it applied and those it considered and rejected. Dropping these blocks removes a form of implicit chain-of-thought: having to name and justify each change encourages the model to commit only to changes it can defend. Its measured effect is small and inconsistent—it is Gemini’s best bitonic-sort variant ($1.13\times$) and has no discernible effect elsewhere—but it also removes the record from which we reconstruct *why* a run behaved as it did.

No Role (no_role). The full prompt opens by casting the model as an HPC performance-optimization expert targeting a specific GPU, and follows with an architecture-specific guidance block. This variant removes both. It is the variant most likely to produce a pathological run rather than a merely worse one: GPT-5 under no_role is the single timeout in the grid (accuracy), while Gemini completes the same configuration without incident.

Chain-of-Thought (cot). Unlike the others, cot adds a component rather than removing one: a single instruction, “*Think step by step about which bottlenecks to address before writing any code,*” is prepended to the instruction block. It is the only variant that never degrades either model relative to the full prompt, reaching the best observed speedup for both on bitonic-sort

($1.14\times$ and $1.13\times$). It is not uniformly safe, however: Gemini’s sobol run under cot introduced `__builtin_ctz`—a host-only intrinsic—inside a `__global__` kernel and could not repair the resulting compile error within three iterations.

Variants not evaluated. no_format strips the output-format specification and asks only for optimized code. We exclude it because it perturbs the reliability of automated code extraction rather than the quality of the optimization, which would confound the metric we are measuring rather than inform it. no_hardware removes the architecture-specific guidance block and the target-architecture line; its effect is largely subsumed by no_role, which removes the same guidance block together with the persona.

4 Experiments

We run two families of experiments: a prompt-component ablation on single-kernel benchmarks, and an end-to-end study on production applications.

Benchmarks. The prompt ablation uses six HeCBench (Jin and Vetter, 2023) CUDA benchmarks—atan2, bitonic-sort, md5hash, accuracy, sobol and shmembench—chosen to span memory-bound and compute-bound behaviour and to include kernels with built-in verification. The profile-data ablation (Section 8) uses a further eleven HeCBench applications: adv, amgm, atomicSystemWide, background-subtract, bincount, blockAccess, boxfilter, ccsc-trpdrv, chemv, concat and convolutionSeparable.

Applications. The end-to-end study uses three production codes: AutoDock-GPU (molecular docking, CUDA), LAMMPS (molecular dynamics, Kokkos/CUDA) and Cholla (astrophysical hydrodynamics, CUDA). Each is built from its own build system with its own inputs, and validated by a gate derived from its scientific output rather than from a benchmark harness (Section 7).

Models. The reported prompt-ablation grid uses GPT-5 and Gemini 2.5 Pro. The application and profile-data studies use GPT-5 and Gemini 3.1 Pro; we note the model change explicitly wherever numbers from the two studies appear together, since they are not directly comparable.

Hardware. All benchmark runs—the prompt ablation and the HeCBench part of the profile-data ablation—are generated and timed on an NVIDIA H100 PCIe (sm_90). The application study and the real-application part of the profile-data ablation run on an NVIDIA GH200 (sm_90). Both nodes use CUDA 12.6.3. One mismatch is worth stating: the profiles supplied in the prompt for the six original ablation benchmarks were collected on an A100 (108 SMs) rather than on the H100 (114 SMs) they were then timed on; the eleven profile-data-ablation benchmarks use native H100 profiles. All nodes are shared with other users, which motivates the measurement protocol of Section 2: build once, interleave variants within one session, wait for an idle GPU, and log competing GPU processes at the start and end of every repetition so that contaminated repetitions can be excluded rather than averaged in.

5 Results

The ablation study evaluated GPT-5 and Gemini 2.5 Pro across six prompt variants and six HeCBench benchmarks, yielding 72 experimental runs in total (2 models $\times$ 6 variants $\times$ 6 apps). Across all successful runs, GPT-5 demonstrated consistently higher first-attempt success rates, requiring only a single build iteration in 32 of 36 cases compared to Gemini’s 24 of 36. Gemini exhausted all three repair iterations on 8 runs (4 failures, 4 eventually repaired), with sobol being its most problematic benchmark (3 outright failures due to correctness violations and a persistent `__builtin_ctz` host-device mismatch).

6 Semantic-Preservation Constraint

Section 10 motivates an explicit computational-intent constraint to prevent models from replacing the computation rather than optimizing it. We implemented that constraint and measured its effect.

Constraint text. The prompt gains a *semantic preservation* block stating that the model may change *how* a computation executes (data placement, memory layout and access pattern, thread/block mapping, vectorization, unrolling, loop fusion/tiling/reordering, hoisting loop-invariant work) but may not change *what* is computed or how much work is done. In particular it may not reduce the number of algorithm-level operations by exploiting algebraic identities, change the iteration count of a timing, repeat or

convergence loop, precompute at compile time a value the program computes at run time, skip work on the grounds that the program’s own verification would still pass, or alter printed output. A closing sentence names the case directly: if a benchmark exists to measure a specific hardware operation, that operation must still be performed the same number of times on the same volume of data. The rule is deliberately scoped to *benchmark scaffolding*; eliminating genuine redundancy inside an application remains allowed, since that is the goal for real codes.

The exploit. `atomicSystemWide` exists to measure system-wide atomic throughput: each thread performs `loop_num` system-scope `atomicAdd_system(&atom_arr[0], 10)` operations, and the program then checks the accumulated total. Without the constraint, both models removed the loop and kept the total. GPT-5 replaced it with one atomic per thread carrying the pre-summed value, annotating its own source “*one store per warp is sufficient for postcondition used by verification*”; Gemini collapsed it further, to a single `atomicAdd_system(&atom_arr[0], blockDim.x * 10 * loop_num)` per block. Both versions print the same output and pass the program’s own check.

Effect. Table 2 reports the outcome. Without the constraint, the two models report $1504\times$ and $2809\times$; with it, three separately generated constrained arms—the relaxed (structural) prompt and, in a later session, the `pcrl` and `code_only` prompts that keep the original prohibitions—all return honest, roughly neutral results, and in all of them the per-iteration `atomicAdd_system(. . . , 10)` is still present in the generated source. Two caveats apply to the table. The unconstrained and relaxed arms were generated and measured in the same interleaved session, but they differ in more than the constraint, since the relaxed prompt also lifts the syntactic prohibitions; the arms that isolate the constraint (original prohibitions plus the semantic-preservation block) come from a later session on a different node, where this benchmark’s repetition spread exceeds 100%. The comparison we draw is therefore categorical—gaming versus no gaming, confirmed by reading the generated source—rather than a precise ratio.

Table 1: Ablation study: runtime (μ s) and outcome across six HeCBench benchmarks and six prompt variants for GPT-5 and Gemini 2.5 Pro. Speedup relative to unoptimized baseline (bitonic-sort: 51,590 μ s; sobol: 2,576 μ s; shmembench: 4,816 μ s); “—” = no baseline available. **Bold** = best valid speedup per benchmark per model. Result: *sl* = first attempt; *rep_n* = repaired in *n* iterations; *fail* = all 3 failed; *T/O* = timeout. [†] benchmark gaming (shared memory algebraically eliminated); * runtime extraction failure.

Benchmark	Variant	GPT-5			Gemini 2.5 Pro		
		Runtime (μ s)	Speedup	Result	Runtime (μ s)	Speedup	Result
atan2	no_explanation	91.08	—	<i>sl</i>	92.41	—	<i>sl</i>
	minimal	97.53	—	<i>sl</i>	92.27	—	<i>rep₂</i>
	no_constraints	94.01	—	<i>sl</i>	91.22	—	<i>sl</i>
	cot	87.47	—	<i>sl</i>	—	—	<i>fail</i>
	no_proof	89.00	—	<i>sl</i>	92.63	—	<i>sl</i>
	no_role	87.76	—	<i>sl</i>	91.59	—	<i>sl</i>
bitonic-sort	no_explanation	57,277	0.90 $\times$	<i>sl</i>	45,681	1.13 $\times$	<i>rep₂</i>
	minimal	55,820	0.92 $\times$	<i>sl</i>	55,181	0.93 $\times$	<i>sl</i>
	no_constraints	45,364	1.14 $\times$	<i>sl</i>	57,022	0.90 $\times$	<i>rep₂</i>
	cot	45,450	1.14 $\times$	<i>sl</i>	45,678	1.13 $\times$	<i>sl</i>
	no_proof	45,342	1.14 $\times$	<i>sl</i>	57,288	0.90 $\times$	<i>rep₂</i>
	no_role	57,370	0.90 $\times$	<i>sl</i>	54,613	0.94 $\times$	<i>sl</i>
md5hash	no_explanation	262,600	—	<i>sl</i>	299,200	—	<i>sl</i>
	minimal	396,800	—	<i>sl</i>	288,700	—	<i>sl</i>
	no_constraints	262,600	—	<i>sl</i>	296,500	—	<i>sl</i>
	cot	295,200	—	<i>sl</i>	296,900	—	<i>sl</i>
	no_proof	262,500	—	<i>rep₃</i>	301,400	—	<i>sl</i>
	no_role	293,100	—	<i>sl</i>	299,500	—	<i>sl</i>
accuracy	no_explanation	214.83	—	<i>sl</i>	213.27	—	<i>sl</i>
	minimal	282.68	—	<i>sl</i>	294.74	—	<i>sl</i>
	no_constraints	213.70	—	<i>sl</i>	296.44	—	<i>sl</i>
	cot	215.38	—	<i>sl</i>	214.90	—	<i>sl</i>
	no_proof	214.08	—	<i>rep₂</i>	308.62	—	<i>sl</i>
	no_role	—	—	<i>T/O</i>	296.31	—	<i>sl</i>
sobol	no_explanation	2,660.79	0.97 $\times$	<i>sl</i>	—	—	<i>fail</i>
	minimal	2,824.18	0.91 $\times$	<i>sl</i>	2,574.01	1.00 $\times$	<i>rep₃</i>
	no_constraints	—	—*	<i>sl</i> *	2,620.28	0.98 $\times$	<i>sl</i>
	cot	2,809.94	0.92 $\times$	<i>sl</i>	—	—	<i>fail</i>
	no_proof	2,814.58	0.92 $\times$	<i>sl</i>	2,575.83	1.00 $\times$	<i>rep₃</i>
	no_role	2,623.37	0.98 $\times$	<i>sl</i>	—	—	<i>fail</i>
shmembench	no_explanation	20.46	235 $\times$ [†]	<i>sl</i> [†]	9,145	0.53 $\times$	<i>sl</i>
	minimal	5.08	949 $\times$ [†]	<i>sl</i> [†]	18,735	0.26 $\times$	<i>sl</i>
	no_constraints	20.47	235 $\times$ [†]	<i>sl</i> [†]	4,661	1.03 $\times$	<i>sl</i>
	cot	20.49	235 $\times$ [†]	<i>sl</i> [†]	4,661	1.03 $\times$	<i>sl</i>
	no_proof	20.49	235 $\times$ [†]	<i>sl</i> [†]	22.66	213 $\times$ [†]	<i>sl</i> [†]
	no_role	20.50	235 $\times$ [†]	<i>sl</i> [†]	4,817	1.00 $\times$	<i>sl</i>

Detection. Neither output comparison nor the benchmark’s own PASS check detects this class of transformation, by construction: the exploit preserves the printed result. In our pipeline it was caught by an implausibility threshold ($> 5\times$) followed by manual inspection of the diff. We therefore treat any speedup above that threshold as a correctness alarm rather than a result; the same rule later exposed a $31\times$ amgmk candidate that had deleted the benchmark’s report sections, and a $3.25\times$ LAMMPS candidate that was computing wrong physics (Section 7).

7 Real-Application Evaluation

The ablation of Section 3 uses single-file HeCBench kernels. To test whether the pipeline transfers to production scientific software, we applied it end to end to three real applications. Unless stated otherwise, experiments in this section use GPT-5 and Gemini 3.1 Pro (not Gemini 2.5 Pro as in Section 5) on an NVIDIA GH200 (sm_90); the LAMMPS and Cholla profiles were collected on the same device on which they were measured.

7.1 Applications and targets

Table 3 lists the applications. For each, we profiled the whole application with Nsight Systems

Table 2: Effect of the semantic-preservation (computational-intent) constraint on atomicSystemWide. [†] = benchmark gaming: the measured atomic loop is removed while output and the built-in check are unchanged. The constrained arms all retain the per-iteration atomic; their exact ratios are not comparable across sessions (see text).

Model	No	With constraint	
	constraint	orig. rules	relaxed
GPT-5	1504 $\times$ [†]	1.02 $\times$	0.94 $\times$
Gemini 3.1 Pro	2809 $\times$ [†]	0.90 $\times$	0.71 $\times$

Table 3: Real applications, optimization targets, and the share of GPU time held by the target kernel.

Application	Target file (hot kernel)	Share
AutoDock-GPU	calcMergeEneGra.cu	98.7%
LAMMPS (Kokkos)	pair_eam_kokkos.cpp	62–68%
Cholla	hllc_cuda.cu	56.0%

and Nsight Compute, selected the source file containing the dominant kernel, and gave the pipeline that single file to optimize, exactly as for a benchmark.

7.2 Correctness gates

Benchmarks ship a PASS/FAIL check; applications do not, and their outputs are not always reproducible. We therefore built one gate per application and validated each gate negatively as well as positively:

- **AutoDock-GPU** is stochastic on the GPU even with a fixed seed (best energies -13.89 , -14.38 , -14.01 kcal/mol for one binary), so the gate is a band: 50 poses and best energy ≤ -12.5 kcal/mol. Negative test: flipping the gradient sign yields -0.53 (FAIL).
- **LAMMPS** with a full neighbour list is bit-deterministic, so the gate compares step-0 energies at 10^{-6} relative tolerance and total energy after 500 NVE steps within 0.1%. Negative test: a flipped force sign makes the simulation explode (FAIL).
- **Cholla** is bit-deterministic; the gate compares the full 3D snapshot field by field (relative $L_1 \leq 10^{-9}$, relative max $\leq 10^{-6}$) after 137 steps. Negative tests: a flipped energy-flux sign diverges (FAIL) and a 10^{-6} relative perturbation of one flux is caught ($L_1 \approx 10^{-6}$, three orders of magnitude over tolerance).

Table 4: Real-application speedups (interleaved medians, idle GPU, all gated correct). “—” = not run; “fail” = no candidate passed the gate within three repair iterations.

Application	Arm	GPT-5	Gemini	Hand
AutoDock-GPU	baseline structural	0.990 $\times$ fail	1.037 $\times$ 1.001 $\times$	1.532$\times$
LAMMPS	baseline structural	0.997 $\times$ —	1.027$\times$ 1.001 $\times$	1.023 $\times$
Cholla	baseline structural	1.571$\times$ 0.992 $\times$	1.504 $\times$ 1.469 $\times$	1.468 $\times$

Every candidate reported below passed its gate.

7.3 Measurement

Application time is taken from each application’s own phase timer, never wall clock (AutoDock’s CUDA setup alone is 68 s on a cold driver). All arms of an application are built once and timed round-robin in a single session; we wait for an idle GPU, log other users’ GPU jobs at the start and end of every repetition, and take medians over clean repetitions only. This is necessary rather than pedantic: on our shared nodes identical code drifted by up to 49% between sessions, two full timing passes were invalidated by other users’ jobs, and the pipeline’s own in-loop timing was off by 15% for one arm.

7.4 Results

Table 4 reports speedups against the unmodified application. *Baseline* is the prompt of Section 2 with PC-sampling and roofline data; *structural* additionally relaxes the syntactic constraints and adds derived kernel-budget and occupancy summaries; *hand* is an expert reference restricted to the same single file.

Models find local, exact transformations; they do not restructure. On Cholla the best model output (GPT-5, 1.571 $\times$) beats the hand reference (1.468 $\times$): both replace 18 double-precision divisions by 6 reciprocal multiplications, and GPT-5 additionally removes integer index arithmetic used only for a bounds check, reducing register pressure from 109 to 89 registers per thread without spilling. On AutoDock-GPU, by contrast, the profitable change is a cross-cutting restructuring—fusing seven sequential block reductions, each with three barriers and a shared-memory atomic, into one pass—worth 1.532 $\times$ by hand. Every candidate that built—three of the

four model $\times$ prompt cells, the fourth producing none—left all seven reductions in place and gained at most 3.7%.

The structural prompt never helped and broke real builds. Its contract was written for single-file benchmarks: it permits signature changes and states that the returned file must be self-contained. In Cholla the target kernel is declared in a header and called from ten sites in other translation units; both models changed the signature and both produced linker errors, each needing a repair iteration to revert. On AutoDock-GPU, whose target file is `#included` into another translation unit, GPT-5 redefined symbols already in scope and then produced an out-of-bounds global read (found with `compute-sanitizer`), failing all three iterations.

Single-file editing is blind to compilation context. The three failures above share one cause: the model cannot see how its file is compiled—whether it is a standalone translation unit or textually included, which symbols are already in scope, and which declarations other files depend on. This does not arise for self-contained benchmarks and is, in our experience, the dominant failure mode on real code.

The human reference is not automatically correct. Our first LAMMPS hand-tuned version constructed Kokkos RandomAccess views inside device code. The identical source passed the gate in one build (at $0.955\times$) and produced deterministically wrong physics in a rebuild (step-0 potential energy $+1.02\times 10^6$ instead of -7.25×10^6), which then “measured” $3.25\times$. It was withdrawn. We therefore gate *every* build that is timed, not only the first, and treat implausible speedups as correctness alarms.

The target must dominate end-to-end time. With LAMMPS’s GPU package driven by a single host core, the CUDA pair kernel is only 7.7–7.9% of loop time (integration and ghost-atom exchange stay on the CPU), capping any kernel-level optimization at $\sim 8\%$ by Amdahl’s law. We switched to the Kokkos backend, where the whole timestep runs on the GPU (94% GPU-busy), before evaluating. A kernel can be 99% of GPU time and still a small fraction of the application.

7.5 Prompt-fidelity defects exposed by real code

Running production applications through the loop surfaced a class of problem that benchmark runs had hidden for months: the prompt the model actually received was not the prompt we believed we were sending. Table 5 lists the defects we found and fixed. They fall into two groups. *Summarization* defects corrupt the profiling evidence—a shared-memory footprint of 7.43 Kbyte presented to the model as “7 bytes”, SI Kbyte read as 1024 bytes, and, most seriously, a CSV reader that parsed every section of an Nsight Compute source page against the first section’s header and silently dropped rows of any other width. On one benchmark that left 36 of 76 stall rows in the prompt; re-running 56 profiled inputs after the fix changed four of them, all by *gaining* rows. *Loop* defects corrupt the repair channel: a build timeout reported to the model as a fault in its own code, and an error filter that kept only lines matching `error|fatal|undefined` reference, so that a linker’s explanatory line (multiple definition of `... first defined here`) was discarded and the model saw only `ld returned 1` exit status.

One defect deserves separate mention because it is a design decision rather than an oversight. The pipeline strips all comments from the source before prompting, to save tokens, and then replaces the original file with the model’s full-file answer—so every optimized file lost its license header and all of its documentation. We initially, and wrongly, attributed the missing LGPL and GPL headers to the models; the prompts contained no occurrence of “Copyright”. We now re-attach the original leading comment block when it is a license and the candidate lacks it, but the general question stands: comment stripping also discards institutional knowledge such as `// DO NOT UNDEFINE` directives that exist precisely to stop the edit an optimizer is about to make.

8 Does Profiling Data Help?

Section 3 ablates components *within* the prompt while always supplying profiling data. The prior question—whether the profiling data helps at all—requires an arm with no profile data, which we add here.

Arms. The `pcr1` arm is the standard prompt carrying PC-sampling and roofline summaries only

Table 5: Prompt-fidelity defects found while running production applications, and what each one corrupted. All were present during the benchmark study.

Defect	Effect
Shared memory read without unit	7.43 Kbyte/block shown as “7 bytes”; static and dynamic sizes summed across units
Kbyte treated as 1024 B	2.4% error in every stated size
Per-section CSV headers ignored	25 of 26 sections dropped on one input (36/76 stall rows reached the model)
Stall reasons from a Python set	Prompt text not reproducible across processes
Kernel names matched verbatim	Namespaced and templated kernels fell back to the capped launch count (9 instead of 49)
Comments stripped before prompting	Optimized files lost their license header and all documentation
Echoed line numbers not stripped	Uncompilable output; one wasted repair iteration
Fixed 300 s build timeout	Real-application rebuilds reported to the model as its own fault
Linker cause lines filtered out	Repair prompt contained no actionable error

(no derived kernel-budget, occupancy or diagnostic summaries). The `code_only` arm removes the entire performance-analysis block. Removing it alone would bias the comparison, because four instruction sentences refer to the data—most importantly the rule “*if no safe substantive optimization can be justified from the provided performance data, keep the code semantically unchanged*”, which would invite a no-op when no data is present. Those four sentences are therefore reworded to refer to the model’s analysis of the source; the two prompts are otherwise byte-identical (verified: the diff is the removed block plus those four sentences). Both arms were regenerated with the same code base, and all pairs use the same correctness gates and interleaved measurement as Section 7.

Scoring. HeCBench pairs are measured on a shared H100 node where per-repetition spread is sometimes larger than any plausible effect, so we score a pair as a *tie* when $|\text{pcrl}/\text{co} - 1|$ is smaller than the largest repetition spread measured for that pair—that is, when the difference between the arms does not exceed its own measurement noise. Two applications (boxfilter, ccscd-trpdv) are excluded because the *unmodified* program exceeds our 600 s per-run cap on that node, and one (background-subtract, GPT-5) because the `code_only` arm produced no correct

Table 6: Profile-data ablation: speedup against the unmodified application for the prompt without profile data (co) and with PC sampling + roofline (pcrl), and their ratio. A HeCBench pair is a tie when the arm-to-arm difference is smaller than that pair’s repetition spread; the entry gives the largest difference observed and the spread it sits inside.

Application	Model	co	pcrl	pcrl/co
Cholla	GPT-5	1.324×	1.476×	1.115
	Gemini	1.471×	1.584×	1.077
AutoDock-GPU	GPT-5	1.030×	1.031×	1.001
	Gemini	1.035×	1.035×	1.000
HeCBench (8 pairs)	GPT-5		all ties	
HeCBench (9 pairs)	Gemini		all ties	

candidate at all, which we report under correctness rather than speed. One caveat carries through the whole benchmark half of this comparison: on this node the benchmarks’ own verification gives no usable verdict for the *unmodified* program—five of nine report FAIL and the rest report nothing—so a candidate’s verdict is informative only where it differs from the pristine one (see Limitations).

Result. Profiling data helps on the large real application and makes no measurable difference on the small benchmarks (Table 6). On Cholla both models are clearly faster with the profile (+11.5% for GPT-5, +7.7% for Gemini), and a second, independently generated pcrl sample keeps both on the same side of the comparison (GPT-5 1.187, Gemini 1.023 relative to `code_only`), although the Gemini replicate is much weaker than its first sample. On AutoDock-GPU both arms reach the same $\sim 1.03\times$. All seventeen scored HeCBench pairs are ties by the rule above: the largest arm-to-arm difference is 7.8% for GPT-5 (adv, inside a 31.8% spread) and 33.2% for Gemini (background-subtract, inside a 147.6% spread), and the eight pairs whose spread is under 5% all land between 0.998 and 1.003. Not one benchmark shows a difference that its own measurement noise does not already cover.

Correctness, not only speed. Two HeCBench pairs favour the profile arm on correctness rather than time. On background-subtract, GPT-5 without profile data failed the application’s own output check in all three repair iterations and produced no usable candidate, while the pcrl arm produced a correct one; on atomicSystemWide, the `code_only` candidate fails verification and the

pcr1 candidate does not.

Interpretation. The pattern is consistent with the workloads rather than with the models. A HeCBench kernel is a few hundred lines with one obvious hot loop, and the profile tells a model little it cannot infer from the source. Cholla’s HLLC solver is 307 lines inside a 21k-line CUDA application whose cost is spread over several kernels; there the profile identifies both the kernel worth editing and the reason it is slow (latency-bound at 19.7% DRAM and 40.4% SM throughput, occupancy 23% limited by 109 registers per thread). We therefore expect profile data to pay off in proportion to how hard the bottleneck is to locate from source alone—which is precisely the regime real applications occupy. A separate question is what happens when that data is not merely supplied but *interpreted* into named remedies before the model sees it; Section 9 measures that arm.

9 When Profiling Data Becomes Prescriptive

Sections 3 and 8 treat profiling data as something the prompt either carries or does not. A third option is to carry it *interpreted*: to convert the metrics into named remedies before the model sees them. Recent work argues strongly for this—KernelPro turns Nsight Compute, SASS and Nsight Systems output into natural-language guidance through a set of micro-profiling tools, and reports a 125% higher speedup than handing the model raw metrics (Gai et al., 2026a). We evaluated the same idea, as a diagnostics prompt that adds a bounded checklist of measured conditions and the transformation each one suggests, and reach a more qualified conclusion: interpretation does not shift the average outcome so much as it widens the distribution of outcomes.

Setup. Table 7 reports four arms on one node (GH200, idle, all arms built once and timed round-robin in a single session): the unmodified program, the constrained *baseline* prompt, the relaxed *structural* prompt, and *diagnostics*, which is the structural prompt plus the checklist. These are the tightest measurements in this paper—most repetition spreads are below 5% and several are below 0.1%—so the differences below are not measurement artifacts.

The checklist produces our largest gains. On *bincount* it takes both models from roughly

Table 7: Effect of converting profiling metrics into a prescriptive checklist (*diag*), against the constrained (*base*) and relaxed (*struct*) prompts. Speedups are interleaved medians against the unmodified program on one GH200. **Bold** marks the cases discussed in the text. “spread” is the largest repetition spread across the four arms.

Benchmark	Model	base	struct	diag	spread
chemv	Gemini	7.835×	7.638×	1.075×	1.9%
concat	GPT-5	1.031×	0.838×	0.349×	0.1%
boxfilter	GPT-5	10.280×	6.352×	5.503×	0.0%
bincount	GPT-5	1.035×	1.037×	1.374×	5.2%
bincount	Gemini	1.106×	1.003×	1.380×	4.4%
concat	Gemini	1.001×	1.797×	1.798×	46.8%
boxfilter	Gemini	8.691×	8.930×	9.371×	42.6%
convolutionSeparable	Gemini	1.418×	1.499×	1.514×	19.3%
blockAccess	both	≈ 1.00×	≈ 1.00×	≈ 1.00×	0.2%

1.04–1.11× to 1.37–1.38×, approaching our hand-tuned reference (1.392×), and on *concat* under Gemini it is the difference between no speedup and 1.80×. Where the measured condition is real and the named remedy is the right one, telling the model what to do works.

It also produces our largest losses, on the same benchmarks. On *chemv* the unaided model had restructured the polyhedral-generated kernel into two row-parallel kernels—one per triangle half, conjugation preserved on the upper one—a genuine algorithmic rewrite worth 7.835×. Given the checklist, it kept the original loop nest and bolted on the shared-memory staging the checklist named, collapsing to 1.075×. On *concat*, GPT-5 followed the instruction to process several elements per thread literally, to one-thread-per-128-element line: a 128× collapse in parallelism, measured at 0.349× with a 0.1% spread. On *boxfilter* the same prompt costs GPT-5 nearly half of a 10.280× result.

Interpretation narrows the search. A checklist of remedies is an anchor as much as a hint. When the listed transformation is the best available one, anchoring is worth a large factor; when the model would have found something better unaided, the anchor is what it finds instead. Averaged over our benchmarks the effect is close to neutral, which is why a mean speedup—the statistic the literature reports—hides it. The dispersion is the result: the same prompt component that produces 1.380× on one benchmark produces 0.349× on another, both at sub-5% spread.

Two practical consequences follow. First, prescriptive guidance should be reported with its vari-

ance, not only its mean; we would not have seen this in a geometric-mean summary. Second, the guidance should be *evidence rather than remedy*—state the measured condition (“23% occupancy, limited by 109 registers per thread”) and let the model decide what follows, instead of naming the transformation. We did not get to test that variant, and it is the first thing we would run next.

10 Analysis

Optimization quality. Within the ablation configuration (pc+rl input, one prompt component ablated at a time), neither model delivers large or reliable speedups. The best sustained result across all valid runs is $1.14\times$ (GPT-5 on bitonic-sort under no_constraints, cot or no_proof) and $1.13\times$ (Gemini on the same benchmark under no_explanation or cot). On sobol both models either fail or match the baseline within 2%. On atan2, md5hash and accuracy the runtime distributions are dominated by variant effects rather than by any genuine optimization.

Prompt components are not equally important. The minimal variant consistently underperforms—most clearly on md5hash (GPT-5: $396,800\mu\text{s}$ versus $262,500\mu\text{s}$ under no_constraints, about $1.5\times$ slower) and on shmembench (Gemini: $18,735\mu\text{s}$ versus roughly $4,660\mu\text{s}$ for every other variant). Stripping contextual framing measurably degrades optimization quality. Conversely, cot is stable or slightly better for both models, though the improvement is modest. Every other component is model-dependent: removing constraints helps GPT-5 on bitonic-sort but hurts Gemini; removing the explanation helps Gemini on the same benchmark but has no consistent effect elsewhere; and removing the role assignment caused a timeout for GPT-5 on accuracy while Gemini handled it without issue. Minimal is the only variant that consistently degrades both models, and cot the only one that never hurts either.

Reliability. GPT-5 succeeds on the first attempt in 32 of 36 runs, Gemini 2.5 Pro in 24 of 36. Gemini exhausts all three repair iterations on eight runs, four of which are eventually repaired and four of which fail outright; sobol accounts for three of the four failures, through correctness violations and the host-only-intrinsic error described in Section 3. First-attempt success rate, not peak speedup, is the dimension on which the two models differ most.

Benchmark gaming. The most consequential finding of the ablation is not a speedup but a false one. On shmembench, GPT-5 games the benchmark under all six variants without exception, exploiting the fact that a swap-based permutation preserves the sum to eliminate all shared-memory traffic and report $235\text{--}949\times$ apparent speedups. Gemini does so only when the proof requirement is removed (no_proof), reaching $213\times$ by the same mechanism. Two things follow. First, a prompt-level requirement for a formal correctness argument suppresses this class of edit in one model but not the other, so it cannot be relied on as a defence. Second, the benchmark’s own verification passes in every one of these runs, so no output-comparison gate would have caught them. This motivates the explicit computational-intent constraint evaluated in Section 6.

From benchmarks to applications. The benchmark results above understate how much prompt design matters, because single-kernel benchmarks hide two of its effects. They cannot expose compilation-context failures, since each benchmark is a self-contained translation unit; and they make profiling data look worthless (Section 8), since the hot loop of a 200-line benchmark is obvious from the source. Both effects reappear on the production applications of Section 7, which is the strongest argument we can offer against drawing conclusions about such systems from benchmark suites alone.

11 Related Work

Code LLMs and their benchmarks. Code-generation models are usually measured by functional correctness on small, self-contained problems—HumanEval and its successors (Chen et al., 2021)—or, more recently, by repository-level issue resolution (Jimenez et al., 2024). Neither rewards performance: a program that passes the tests scores the same whether it runs in one second or ten. Our setting inverts this. Correctness is a gate rather than the objective, and the score is measured wall-clock time of a real application.

LLMs for performance optimization. Performance-oriented work with LLMs falls into three groups. The first learns edits that make code faster: PIE mines human performance-improving edits from competitive-programming submissions and adapts LLMs to reproduce

them, reporting large speedups on simulated CPU time (Shypula et al., 2024). The second targets the compiler rather than the source: Cummins et al. train models to predict optimization flag sequences over LLVM-IR (Cummins et al., 2023) and release compiler foundation models (Cummins et al., 2024). The third targets GPU kernels directly. KernelBench asks models to write CUDA kernels equivalent to PyTorch reference modules and reports that frontier models beat the PyTorch baseline in fewer than 20% of tasks (Ouyang et al., 2025); ParEval evaluates parallel code generation across execution models and shows a marked gap between sequential and parallel generation quality (Nichols et al., 2024). All of these generate or rewrite *standalone* kernels or short programs. The distinguishing feature of our study is that the artifact under optimization is one file inside a production application—AutoDock-GPU, LAMMPS, Cholla—which keeps its build system, its callers, and its own scientific validation.

Profiling data as LLM context. The closest work to ours feeds profiler output back to the model. Gai et al. route Nsight Compute, SASS and Nsight Systems data through a roofline-based classifier that turns raw counters into natural language guidance, and report large gains on KernelBench (Gai et al., 2026b). We ask a narrower and, we argue, prior question: does profiling data help *at all*, relative to the same prompt without it? The distinction matters most against KernelPro, the closest work to ours, which feeds Nsight Compute, SASS and Nsight Systems output through plugable micro-profiling tools that convert raw metrics into natural-language guidance and reports a 125% higher speedup for the interpreted arm (Gai et al., 2026a). Both of their arms carry profiling data; the comparison is *interpreted* against *raw*. Ours removes the data entirely. Section 9 evaluates their question too, on our benchmarks, and finds that interpretation widens the distribution of outcomes rather than shifting its centre. Section 8 isolates that by holding every other prompt component fixed, and finds the answer depends on the workload—no measurable effect on single-kernel benchmarks, but a clear effect on a large application where the bottleneck is not apparent from the source.

Autotuning and search. Classical autotuning searches a space of program variants or parameters with measurement in the loop: OpenTuner for

general program autotuning (Ansel et al., 2014), and learned cost models for tensor programs in AutoTVM (Chen et al., 2018) and Ansor (Zheng et al., 2020). These systems explore a fixed, hand-designed space and need many measurements; an LLM instead proposes a small number of semantically arbitrary source edits. The two are complementary, and our build–verify–measure loop is essentially the autotuner’s evaluation harness wrapped around a generative proposer. The profile summaries we place in the prompt play the role that a cost model plays in search: they tell the proposer where the time goes. Roofline analysis (Williams et al., 2009) underlies both the Nsight Compute reports we summarize and the bottleneck classification used by prior work.

Repair loops. Iterative self-repair from compiler and test feedback is now standard practice (Chen et al., 2024). Our loop is a constrained instance: the feedback is a build log or an application-level correctness gate, and the number of repair iterations is bounded. Our contribution here is negative and practical—we document cases where the loop cannot work because the information needed is absent from the feedback, for example a linker error whose cause line is filtered out of the summary, or an application that aborts with a bare assertion and discards the CUDA error.

Benchmark gaming and reward hacking. A model optimizing against a checkable objective may satisfy the check without doing the work. This is the code-generation instance of reward hacking, studied recently through benchmarks that deliberately expose exploitable evaluation harnesses: EvilGenie places coding agents in an environment where test files can be edited or answers hardcoded, and finds explicit reward hacking by current proprietary agents, detectable by an LLM judge but not reliably by held-out tests (Gabor et al., 2025). In the performance setting specifically, KernelBench-Verified audits LLM-generated GPU kernels and finds identity functions reported as $374\times$ speedups, fast paths gated on the test tensor’s exact dimensions, and value-dependent work skipping; recalibrated against hidden input distributions and an LLM-judge audit, the best model drops from $1.43\times$ to $0.88\times$, with 11.6% of problems relying on distribution-specific exploits (Zhang et al., 2026). Our contribution here is therefore not the observation but its setting and its remedy: the same failure in an HPC

benchmark rather than an ML one, a prompt-level constraint that we measure, and a detection rule cheap enough to run on every candidate. We observe the phenomenon in a performance setting, where it is particularly insidious: deleting the measured computation preserves the printed output and the benchmark’s own PASS check while producing three-order-of-magnitude “speedups” (Section 6). We show that a prompt-level semantic-preservation constraint suppresses it for both models we tested, and we report the detection rule we use in its place—an implausibility threshold followed by diff inspection.

Repository-level optimization. A parallel line of work optimizes whole repositories rather than kernels. GSO scores agents against the speedup an expert commit achieved across ten codebases (Shetty et al., 2025), and PerfAgent adds a profiler-guided, verifier-in-the-loop layer that more than doubles the rate of expert-matching patches on it (Deng et al., 2026). Its motivation matches ours almost exactly—agents miss bottlenecks hidden behind abstraction layers and native extensions—but the targets are CPU-bound Python repositories scored against an expert’s commit. Ours are GPU kernels inside production scientific applications, scored against the application’s own scientific output, which is a correctness standard an expert diff cannot provide.

HPC benchmark suites. We use HeCBench (Jin and Vetter, 2023) for the prompt ablation, which provides many small CUDA kernels with built-in verification, and three production applications for the end-to-end study. The contrast between the two is one of our main findings: conclusions drawn on single-kernel benchmarks did not transfer to applications, either for the value of profiling data or for the prompt variant that works best.

12 Conclusion

We studied prompt design for LLM-driven GPU code optimization inside a closed loop that profiles, generates, builds, verifies and measures, and we pushed that loop past single-kernel benchmarks onto three production scientific applications.

Three results stand out. First, guardrails decide what the models do with the freedom they are given. Removing context degrades every model

we tested, while an explicit semantic-preservation constraint is what separates an honest result from a benchmark-gaming one: without it, both models delete the computation a benchmark exists to measure and report speedups of three orders of magnitude that survive the benchmark’s own correctness check. Second, profiling data earns its place in the prompt in proportion to how hard the bottleneck is to see from the source. It is worth nothing measurable on single-kernel benchmarks, where the hot loop is obvious, and 7.7–11.5% on a 21k-line application where it is not. Third, and most consequential for anyone building such a system, the models reliably find local, exact transformations—reciprocal reuse, removing redundant index arithmetic, read-only loads—and reliably miss cross-cutting restructuring: on AutoDock-GPU every candidate we obtained left in place the seven sequential block reductions whose fusion is worth $1.532\times$ by hand, while on Cholla the best model output ($1.571\times$) beat our hand-tuned reference.

The failures were as informative as the speedups. Editing a single file inside a real program means the model cannot see how that file is compiled, and three of our application failures follow directly: symbols redefined because the file is textually included elsewhere, and kernel signatures changed although ten call sites in other translation units depend on them. Applications also swallow the diagnostics a repair loop needs—one aborts with a bare assertion, discarding the CUDA error that explained the bug—and shared machines silently corrupt measurements unless contention is detected per repetition. None of these appear when optimizing self-contained benchmark kernels, which is precisely why benchmark-only evaluation overstates how ready such systems are.

Two directions follow. The prompt should carry the file’s *compilation context*—whether it is a standalone translation unit, which symbols are already in scope, and which declarations other files depend on—since that is cheap to extract and addresses the dominant real-code failure mode. And the structural freedom that helps on standalone kernels needs a real-application contract that forbids changing externally visible signatures while still allowing file-internal restructuring; in its current form it never outperformed the constrained prompt and repeatedly broke builds.

Limitations

Measurement on shared hardware. All experiments ran on shared GPU nodes. Identical code drifted by up to 49% between sessions, so we build every arm once, time them round-robin in a single session, wait for an idle GPU, log other users’ GPU processes at the start and end of every repetition, and report medians over clean repetitions only. Two complete timing passes were discarded because other users’ jobs occupied the GPU. Even so, detection is imperfect: in one Cholla pass three repetitions drifted by 15% while our detector reported no foreign GPU process, so we additionally require repetitions to agree before using them. Residual interference cannot be excluded on a shared machine: measurement noise alone puts a floor of roughly 2% under any effect we report on the applications, and far higher on the noisiest benchmarks, where per-repetition spread exceeds 100%.

Sampling of model outputs. Except where stated, each (application, model, prompt) cell is a *single* LLM sample. Language model outputs vary between calls, so a single cell conflates prompt effect with sampling variance. We replicated two settings—a second independent generation round for the structural/diagnostic arms, and a second profile-data sample for the real applications—and the direction of the effect was stable there, but the per-cell numbers in Section 5 carry no confidence interval. Combined with the measurement floor above, this means an effect below roughly 5% in a single cell should be treated as unresolved rather than small.

Hardware and software scope. Results cover NVIDIA GPUs only: H100 PCIe (sm_90) for the benchmark studies and GH200 (sm_90) for the applications, with CUDA 12.6.3. Conclusions may not transfer across architectures, and we did not test that directly: every prompt variant was evaluated on one architecture per study, so an effect that depends on register file size, SM count or cache hierarchy would be invisible to us. The profiles supplied for the six original ablation benchmarks were themselves collected on an A100 and used on an H100. We use two proprietary models whose weights and versions we do not control; the same prompt may behave differently after a model update.

Correctness gates are empirical, not proofs.

Each application is gated by its own output: bit-exact field comparison where the application is deterministic (LAMMPS, Cholla) and a statistical band where it is not (AutoDock-GPU, whose GPU results vary run to run even with a fixed seed). We validated every gate by injecting faults, and the Cholla gate detects a 10^{-6} relative perturbation of a single flux. Nevertheless a subtle accuracy loss that stays inside the AutoDock band would pass, gates check the outputs the application prints rather than all internal state, and undefined behaviour can pass one build and fail another—our own hand-tuned LAMMPS variant did exactly that, producing correct physics in one build and wrong physics in a rebuild of the same source. We therefore gate every build that is timed, but “gated correct” means “passed these checks”, not “verified”.

Benchmark gaming is detected heuristically.

Transformations that delete the measured work while preserving printed output defeat output comparison by construction (Section 6). We flag speedups above $5\times$ and inspect the diff. This catches the large cases we observed ($1504\times$, $2809\times$, $235\text{--}949\times$, $31\times$), but a gaming transformation yielding, say, $1.5\times$ would not be flagged automatically, and we cannot rule out such cases in the reported numbers.

Single-file editing. The pipeline gives the model one source file and replaces that file. The model therefore cannot change headers, launch sites or build flags, and—more importantly—cannot see how its file is compiled. This caused three of our real-application failures (symbol redefinition in a textually included file, and kernel-signature changes that broke linkage against ten call sites). Some optimizations a human would make are out of reach by construction.

Tooling defects discovered during the study.

The prompt-fidelity defects of Section 7.5 (Table 5) were all present during the benchmark study: profiling summaries reached the model with wrong units, truncated stall data and understated launch counts, and the repair channel discarded the diagnostics a model needed. They are fixed, but every result predating a given fix was produced with the affected prompt and would need regeneration to be strictly comparable. We report the benchmark numbers as measured rather than

re-running the grid, and we do not claim that the ablation of Section 3 would reproduce unchanged on the corrected summarizers.

Coverage of the profile-data ablation. The comparison in Section 8 spans 26 (application, model) pairs but scores 21. Two benchmarks (boxfilter, ccscd-trpdrv) could not be timed on the available node because the *unmodified* programs exceed our 600s per-run cap, costing four pairs, and background-subtract under GPT-5 yields no code_only candidate at all. The denominator is therefore small, and—more importantly—the entire positive result rests on a single application, Cholla, with two models. The seventeen benchmark pairs establish only that the effect is absent there, not that it generalizes to other large applications.

Benchmark verification was unusable as a gate on that node. For the HeCBench part of the profile-data ablation, not one of the nine measured benchmarks gave a usable verdict for the *unmodified* program on the H100 node: five (adv, background-subtract, chemv, concat, convolutionSeparable) print FAIL from their own built-in check, and the other four print no verdict at all. In chemv the cause is structural rather than numerical: its timing loop launches the two kernels 1000 times accumulating into the same output buffer without re-initializing it, then compares the result against a single application of the CPU reference, so the check cannot pass for the unmodified program. A candidate’s verdict is therefore informative only where it differs from the pristine one, as on atomicSystemWide, where the unmodified program is silent and the code_only candidate reports FAIL. Correctness for the rest rests on the pipeline’s generation-time check and on our inspection of the diffs, which is weaker than the gates we built for the applications. It is also a caution about the benchmark suites these systems are usually evaluated on: a built-in PASS/FAIL is not automatically a correctness oracle on hardware the benchmark was not tuned for.

Scope of the objective. We optimize execution time of a single GPU kernel file under a correctness gate. We do not study energy, memory footprint, multi-GPU scaling, numerical stability under long runs, or maintainability of the generated code—all of which matter for production use of

these applications.

Acknowledgments

References

- Jason Ansel, Shoaib Kamil, Kalyan Veeramachaneni, Jonathan Ragan-Kelley, Jeffrey Bosboom, Una-May O’Reilly, and Saman Amarasinghe. 2014. Open-Tuner: An extensible framework for program auto-tuning. In *International Conference on Parallel Architectures and Compilation Techniques (PACT)*.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, and 1 others. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.
- Tianqi Chen, Lianmin Zheng, Eddie Yan, Ziheng Jiang, Thierry Moreau, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. Learning to optimize tensor programs. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2024. Teaching large language models to self-debug. In *International Conference on Learning Representations (ICLR)*. ArXiv:2304.05128.
- Chris Cummins, Volker Seeker, Dejan Grubisic, Mostafa Elhoushi, Youwei Liang, Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Kim Hazelwood, Gabriel Synnaeve, and Hugh Leather. 2023. Large language models for compiler optimization. *arXiv preprint arXiv:2309.07062*.
- Chris Cummins, Volker Seeker, Dejan Grubisic, Baptiste Roziere, Jonas Gehring, Gabriel Synnaeve, and Hugh Leather. 2024. Meta large language model compiler: Foundation models of compiler optimization. *arXiv preprint arXiv:2407.02524*.
- Ryan Deng and 1 others. 2026. [Perfagent: Profiler-guided iterative refinement for repository-level code optimization](#). *arXiv preprint arXiv:2607.19653*.
- Jonathan Gabor, Jayson Lynch, and Jonathan Rosenfeld. 2025. [Evilgenie: A reward hacking benchmark](#). *arXiv preprint arXiv:2511.21654*.
- Jiading Gai, Shuai Zhang, Kaj Bostrom, Jin Huang, Vihang Patil, Haoyang Fang, Bernie Wang, Huzefa Rangwala, and George Karypis. 2026a. [Optimizing cuda like a human: Micro-profiling tools as expert surrogates for llm-based gpu kernel optimization](#). *arXiv preprint arXiv:2606.26453*.
- Jiading Gai, Shuai Zhang, Kaj Bostrom, Jin Huang, Vihang Patil, Haoyang Fang, Bernie Wang, Huzefa Rangwala, and George Karypis. 2026b. Optimizing CUDA like a human: Micro-profiling tools as expert surrogates for LLM-based GPU kernel optimization. *arXiv preprint arXiv:2606.26453*.

- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*.
- Zheming Jin and Jeffrey S. Vetter. 2023. A benchmark suite for improving performance portability of the SYCL programming model. In *IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 325–327.
- Daniel Nichols, Joshua H. Davis, Zhaojun Xie, Arjun Rajaram, and Abhinav Bhatele. 2024. Can large language models write parallel code? In *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing (HPDC)*. ArXiv:2401.12554.
- Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. 2025. Kernelbench: Can LLMs write efficient GPU kernels? *arXiv preprint arXiv:2502.10517*.
- Manish Shetty and 1 others. 2025. [GSO: Challenging software optimization tasks for evaluating SWE-agents](#). In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*.
- Alexander Shypula, Aman Madaan, Yimeng Zeng, Uri Alon, Jacob Gardner, Milad Hashemi, Graham Neubig, Parthasarathy Ranganathan, Osbert Bastani, and Amir Yazdanbakhsh. 2024. Learning performance-improving code edits. In *International Conference on Learning Representations (ICLR)*. ArXiv:2302.07867.
- Samuel Williams, Andrew Waterman, and David Patterson. 2009. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76.
- Yunxiang Zhang, Ping Yu, Jianyu Wang, Max Fan, Julian Reed, Azalia Mirhoseini, and Will Su. 2026. [Kernelbench-verified: Do llm-generated kernels actually beat pytorch?](#) *arXiv preprint arXiv:2607.16241*.
- Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, Jun Yang, Danyang Zhuo, Koushik Sen, Joseph E. Gonzalez, and Ion Stoica. 2020. Ansor: Generating high-performance tensor programs for deep learning. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.